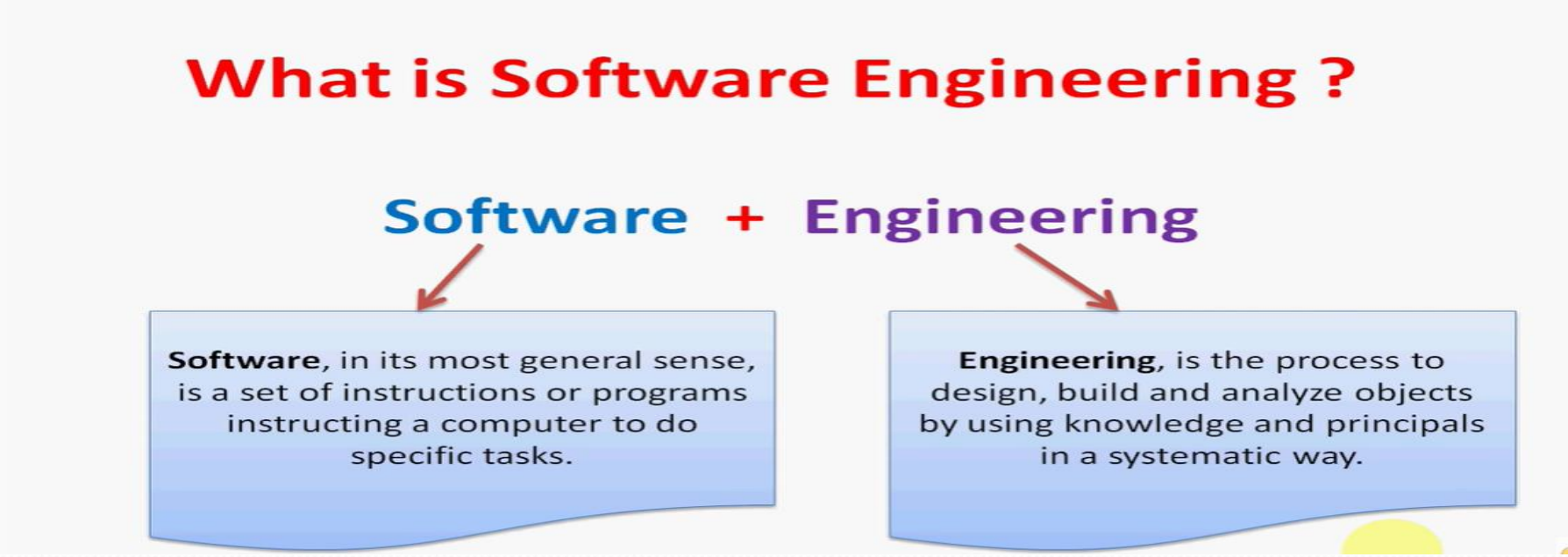
G.H. Raisoni College of Engineering and Management

Department of Information Technology

Unit : 1
Introduction to Software Engineering

Prof. Sagar Mahajan
Prof. Shivkumar Lilhare

UNIT-I Introduction



The process of developing , maintaining a software in a systematic , disciplined manner is called as software Engineering .

Software Engineering is the application of Engineering to the development of software in a systematic Method.

Defining software

- Software is nothing but a collection of computer programs and related documents that are intended to provide desired features, functionalities and better performance.

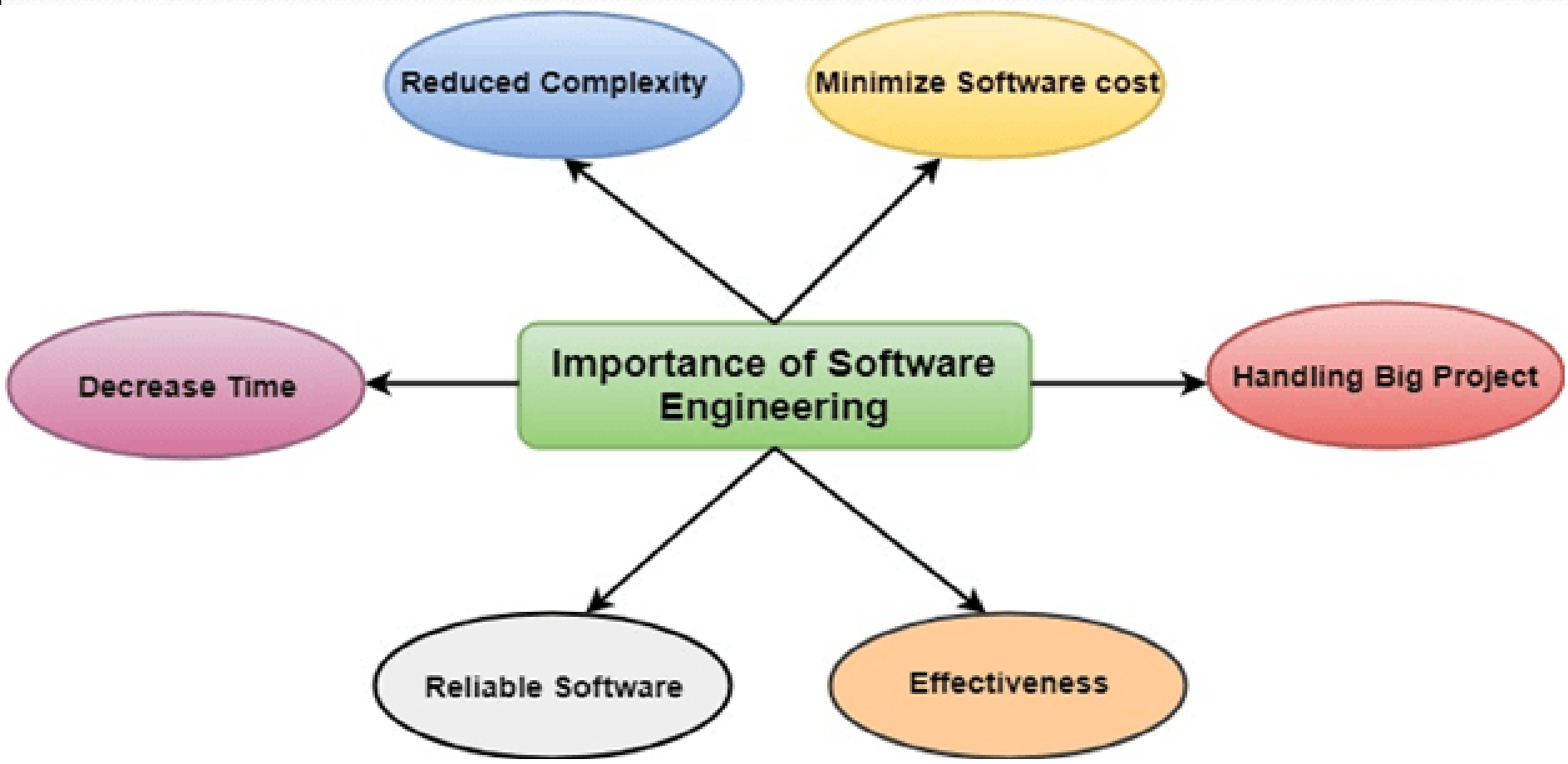
Software product are of:

- **Generic:** used by different customers eg: **Word Processor ,Database Application access,Powerpoint,Web browser,etc.**
- **Custom:** developed for single customer according to specification.eg: **CMS(content Management System),CRM(Customer Relationship Management)**

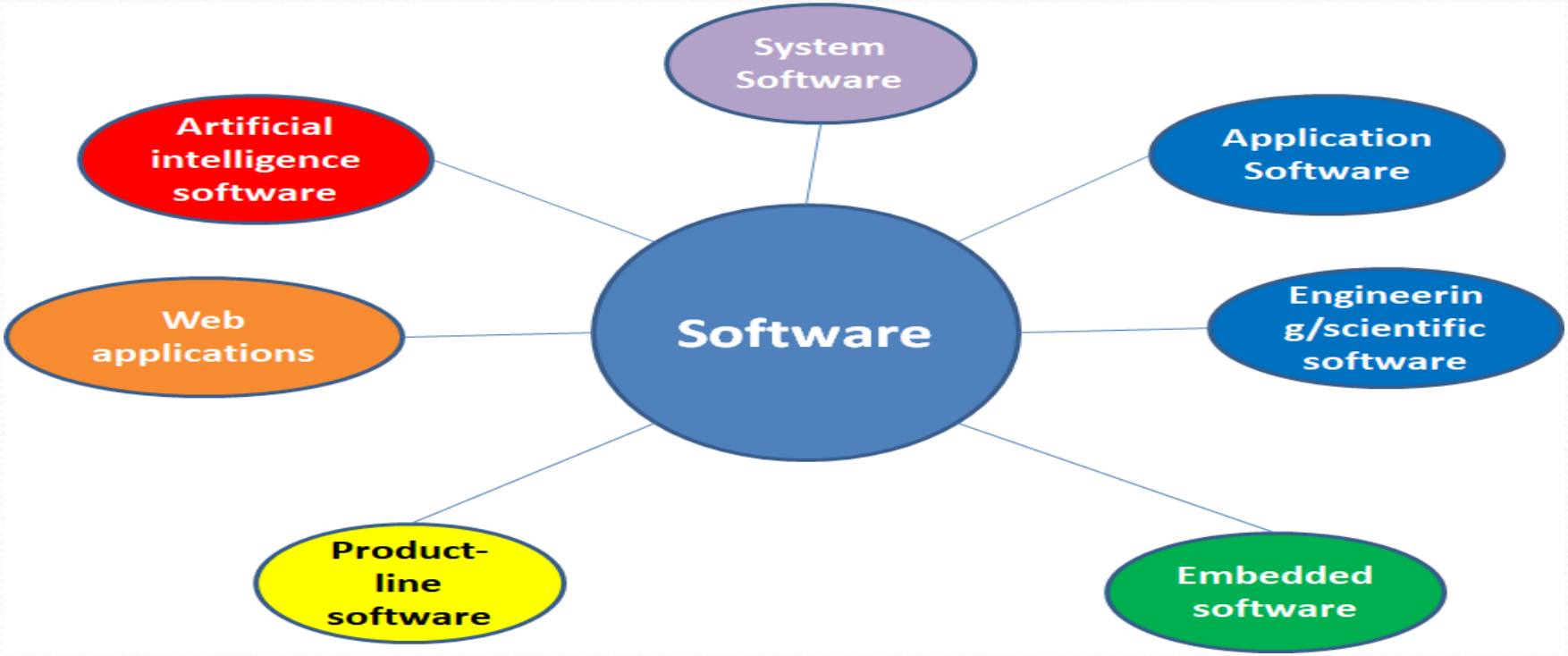
Need of Software Engineering

- Huge Programming
- Adaptability
- Cost
- Dynamic Nature
- Quality Management

Importance of Software Engineering



The Nature Software



Based on a Complex growth of software it can be classified into following categories

1.System Software

This is a collection of programs which are written to service other programs

- Typical programs in this category are compiler , editors & assemblers

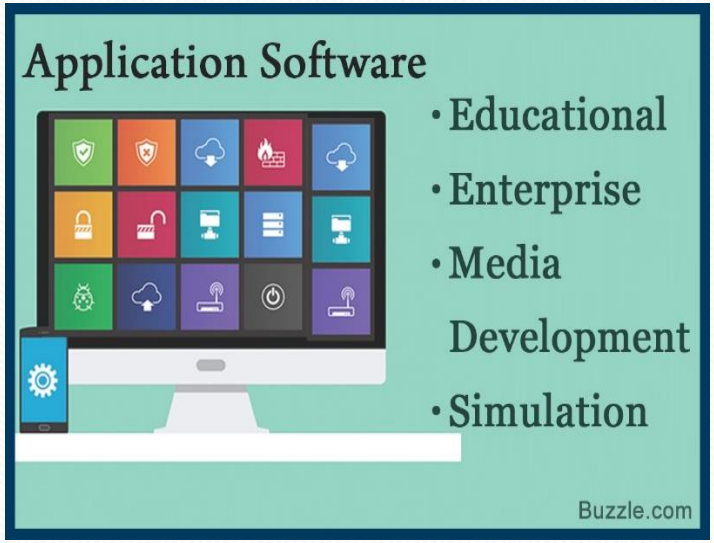
.



2.Application Software :

That are developed for specific business need,

This supported by Database systems.

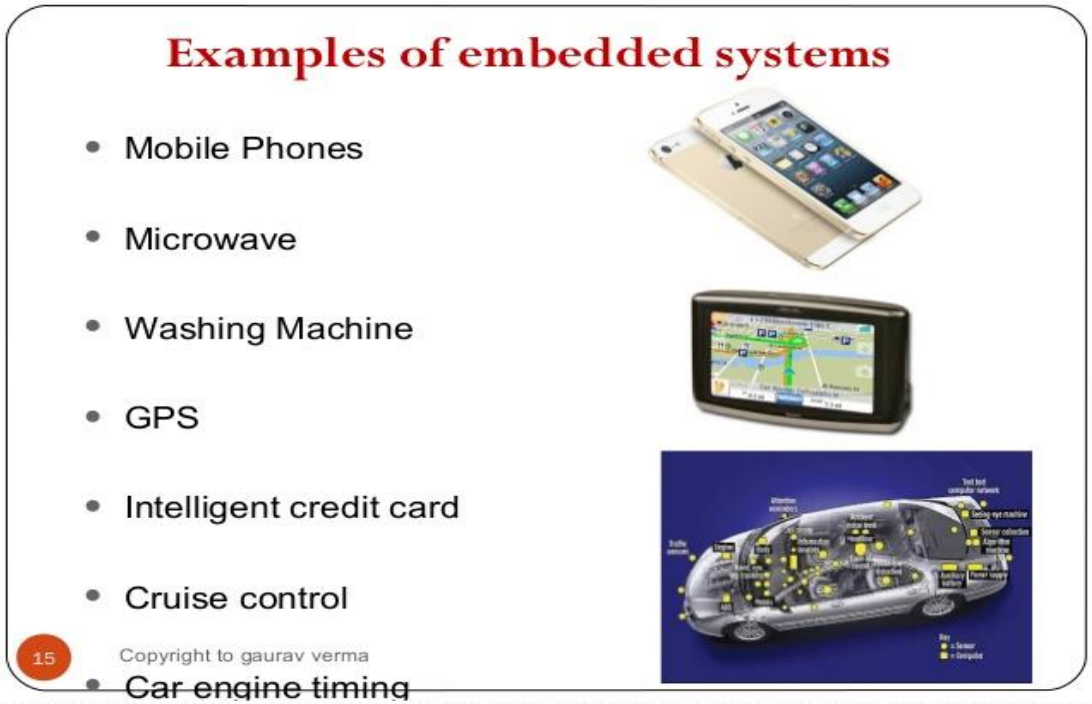


3. Engineering/scientific Software

This is used to facilitate engineering functions and tasks.CAD,System Simulation are examples

4. Embedded Software:

- This type of Software is placed in “Read Only Memory” of the product and control various functions.
- The product could be an aircraft, automobile ,security system, signalling system.



5.Web applications :

- Web application software consists of various web pages that can be retrieved by browser.
- Examples are CGI,HTML,JAVA,PERL,DHTML ,etc.
- It is client-server computer program.

6.Artificial intelligence software:

- This kind of software is based on knowledge based expert systems.
- This Software is useful in Robotics , expert systems , image and Voice recognition.

Software Engineering Practice

Following are Some principles that focus on software engineering Practice.

1.Reason for Existence

2.Keep It Simple

3.Vision

4.The produce is getting consumed

5.Be Open to future

6.Plan for Reuse

7.Think and then act

Software Characteristics

Some software characteristics

I. Software is engineered or developed ,not manufactured.

1. Software development and hardware development are two different activities.
2. A good design is backbone for both the activities.

3. Quality problems that occur **in hardware manufacturing** phase can't be removed whereas, during **software development process** such problems can be rectified.

II. The software doesn't "wear out.

- In early stage of hardware development process the **failure rate** is very high because of manufacturing defects. But after correcting such defects failure rate gets reduced.
- The failure remains constant for some period of time and again start increasing because of **environmental maladies(extreme temp,dusts and vibrations)**
- On the other hand software does not get affected from such environmental maladies. Hence it should have ***an "idealized Curve".*** But due to some **undiscovered errors** the failure rate is high and drops down as soon as the errors get corrected. Hence in failure rate actual curve is shown

Failure Curves for Hardware and Software

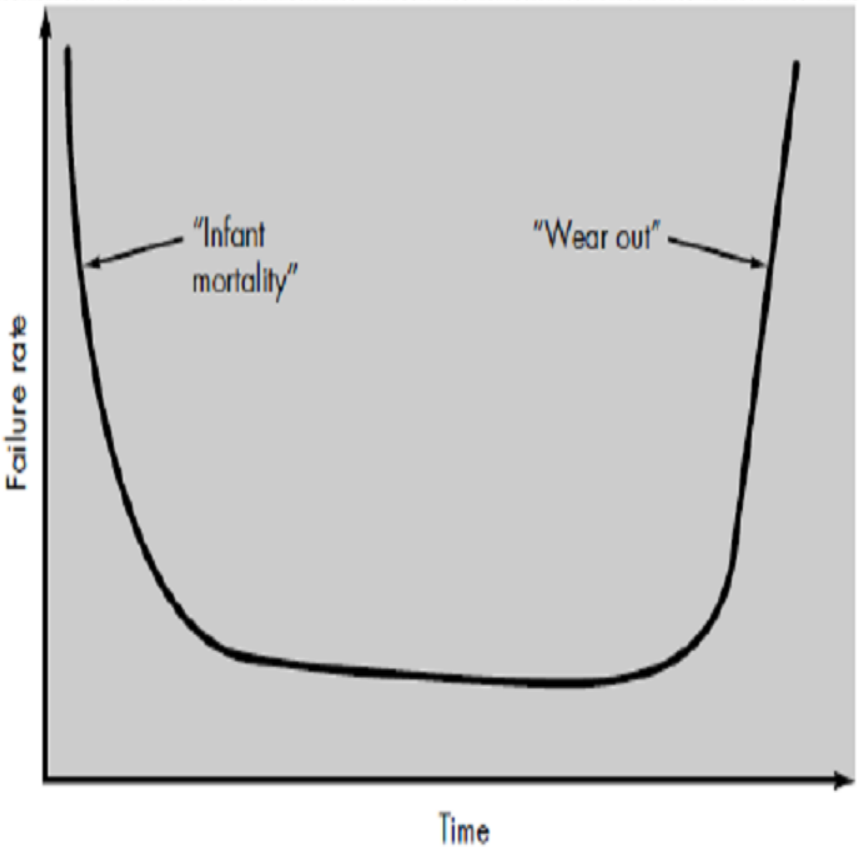


Fig. Hardware curve

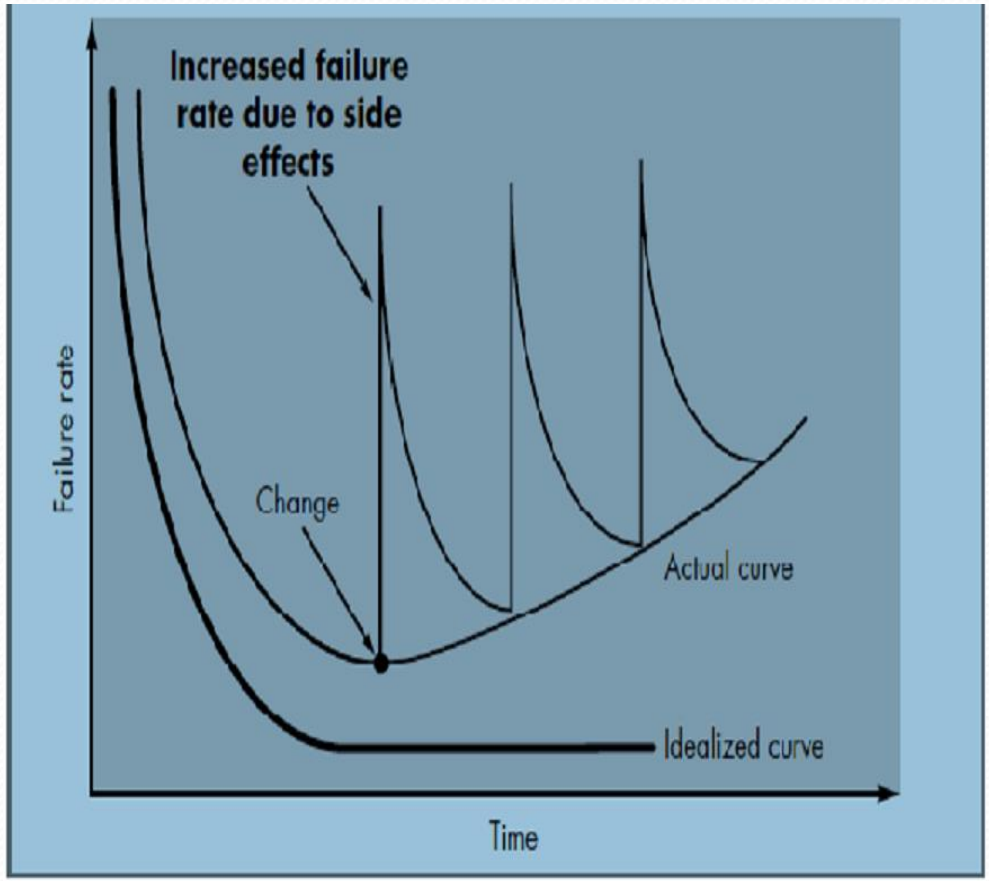


Fig. Software Curve

III. Most software is custom built rather than being assembled from components

- While developing any software product firstly the circuit design with desired functions and properties is created. Then required hardware components such **as ICs, capacitors and registers** are assembled according to design, but this is not possible while developing software product.
- Most software is custom built.
- It is practiced to reuse algorithm and data structures.
- Today software industry is trying to make library of reusable contents. **For example:** In todays's software,**GUI** is built using reusable components such as **message windows, pulldown menus, and many more components.**

Software Engineering Myths

- Software myths are misleading attitudes that have caused serious problems for managers and technical people alike. Software myths propagate misinformation and confusion. There are three kinds of software myths:
- Management Myth
- Customer Myth
- Practitioner's Myth

Management Myths:

Management myths: Managers with software responsibility are often under pressure to maintain budgets, keep schedules from slipping, and improve quality. Following are the management myths:

1.**Myth:** We already have a book that's full of standards and procedures for building software, won't that provide my people with everything they need to know?

Reality: The book of standards may very well exist, but isn't used. Most software practitioners aren't aware of its existence. Also, it doesn't reflect modern software engineering practices and is also complete.

2. **Myth**: My people have state-of-the-art software development tools, after all, we buy them the newest computers.

Reality: It takes much more than the latest model mainframe, workstation, or PC to do high-quality software development. Computer-aided software engineering (CASE) tools are more important than hardware for achieving good quality and productivity, yet the majority of software developers still do not use them effectively.

Customer Myths:

2) Customer myths: Customer myths lead to false expectations (by the customer) and ultimately, dissatisfaction with the developer. Following are the customer myths:

Myth: A general statement of objectives is sufficient to begin writing programs- we can fill in the details later.

Reality: A poor up-front definition is the major cause of failed software efforts. A formal and detailed description of the functions, behavior, performance, interfaces, design constraints, and validation criteria is essential.

Myth: Project requirements continually change, but change can be easily accommodated because software is flexible.

Reality: It is true that software requirements change, but the impact of change varies with the time at which it is introduced. When changes are requested during software design, the cost impact grows rapidly. Resources have been committed and a design framework has been established. Change can cause heavy additional costs. Change, when requested after software is in production, can be much more expensive than the same change requested earlier.

Practitioner's Myths:

3) Practitioner's myths: Practitioners have following myths:

Myth: Once we write the program and get it to work, our job is done.
Reality: Industry data indicate that between 60 and 80 percent of all effort expended on software will be expended after it is delivered to the customer for the first time.
Myth: Until I get the program "running" I have no way of assessing its quality.
Reality: One of the most effective software quality assurance mechanisms can be applied from the inception of a project—the formal technical review.

Software Process

Generic Process Model

- A process is defined as the collection of various activities.
- These activities are performed when some work product is to be created.
- Each of these activities reside within a framework or model that defines their relationship with the process and with one another.

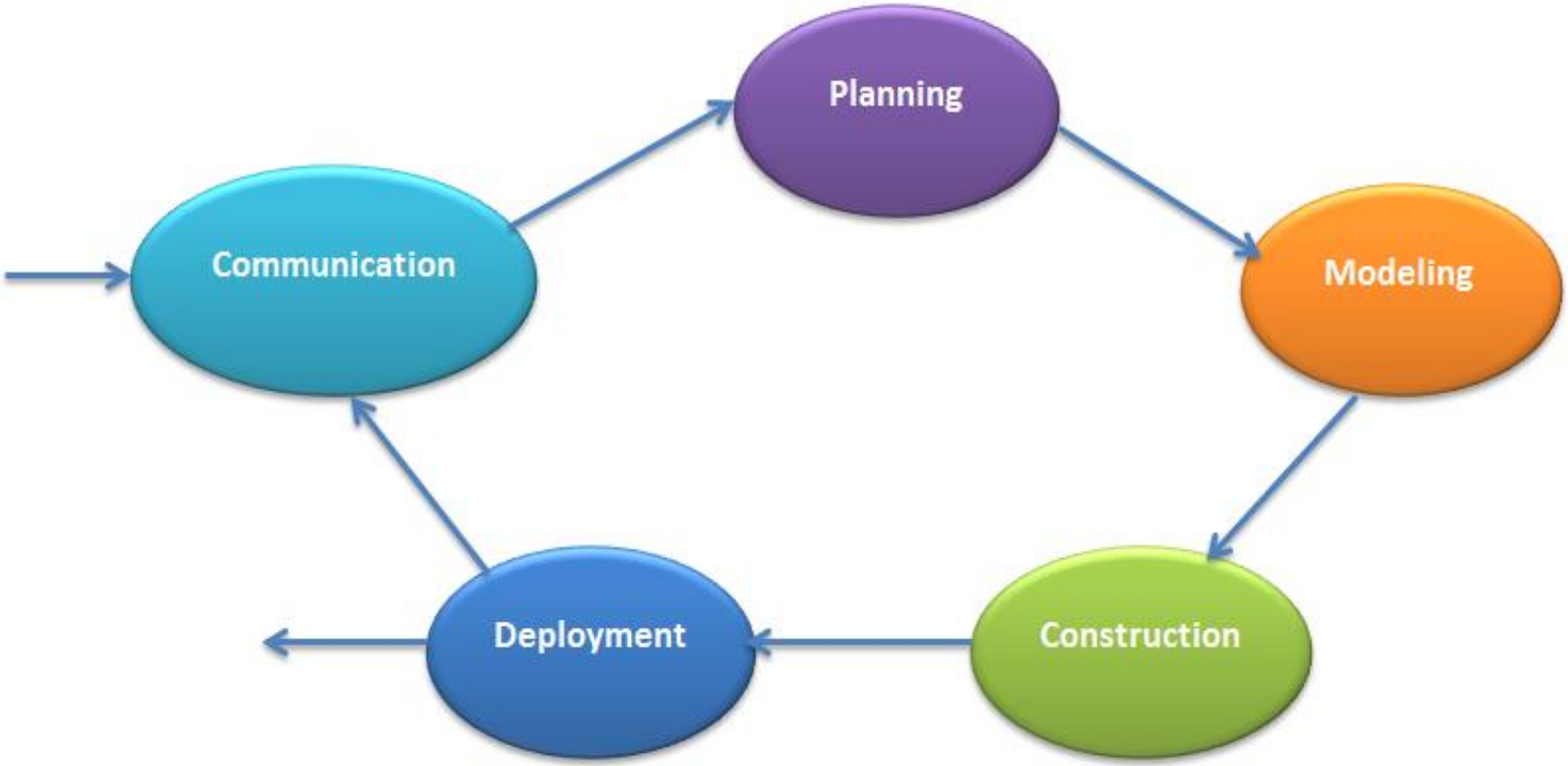


Fig: Generic Process Model

Generic Process Model

There are five generic process framework activities:

1.Communication:

The software development starts with the communication between customer and developer.

2. Planning:

It consists of complete estimation, scheduling for project development and tracking.

3. Modeling:

Modeling consists of complete requirement analysis and the design of the project like algorithm, flowchart etc.

Generic Process Model

4. Construction:

- Construction consists of code generation and the testing part.
- Coding part implements the design details using an appropriate programming language.
- Testing is to check whether the flow of coding is correct or not.
- Testing also check that the program provides desired output.

5. Deployment:

- Deployment step consists of delivering the product to the customer and take feedback from them.
- If the customer wants some corrections or demands for the additional capabilities, then the change is required for improvement in the quality of the software.

Generic Process Model

Umbrella Activities:

A set of umbrella activities—project tracking and control, risk management, quality assurance, and others—are applied throughout the process.

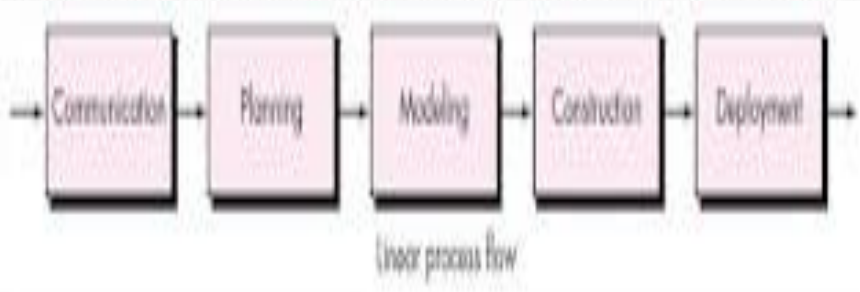
Process flow—describes how the framework activities and the actions and tasks that occur within each framework activity are organized with respect to sequence and time.

Generic Process Model

- A *linear process flow* executes each of the five framework activities in sequence.
- An *iterative process flow* repeats one or more of the activities before proceeding to the next.
- An *evolutionary process flow* executes the activities in a "circular" manner. Each circuit through the five activities leads to a more complete version of the software.
- A *parallel process flow* executes one or more activities in parallel with other activities

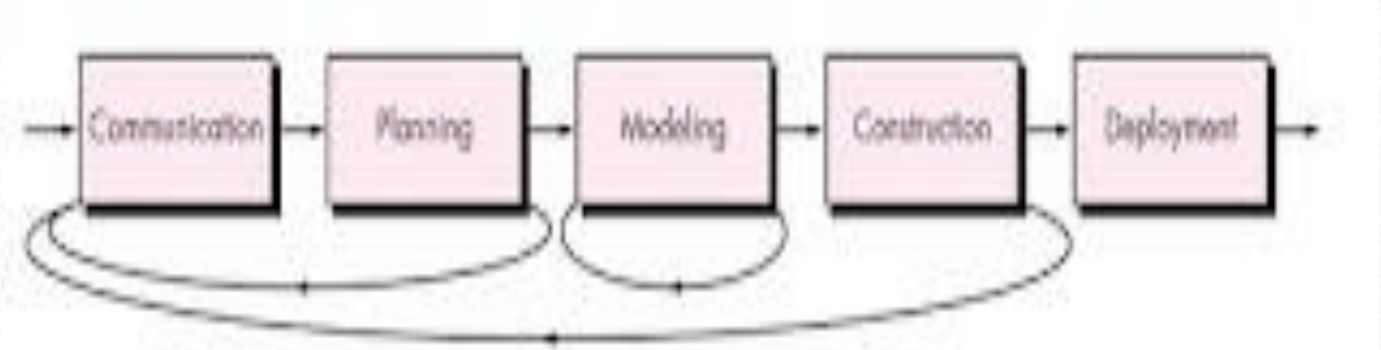
Process Flows

- **Linear Process Flow**: A **linear process flow** executes each of the five framework activities in sequence, beginning with communication and culminating with deployment.



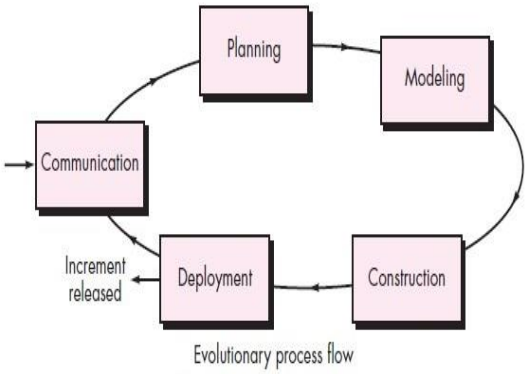
Linear process flow

Iterative Process Flow: An **iterative process flow** repeats one or more of the activities before proceeding to the next.



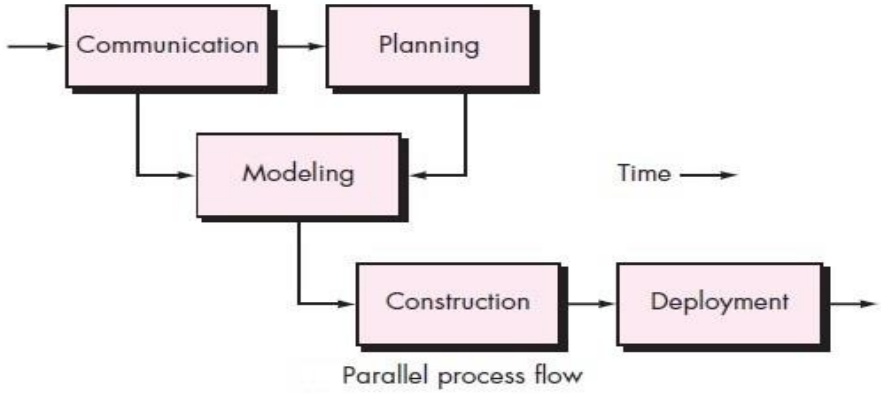
Iterative process flow

- **Evolutionary Process Flow**: An **evolutionary process flow** executes the activities in a "circular" manner.



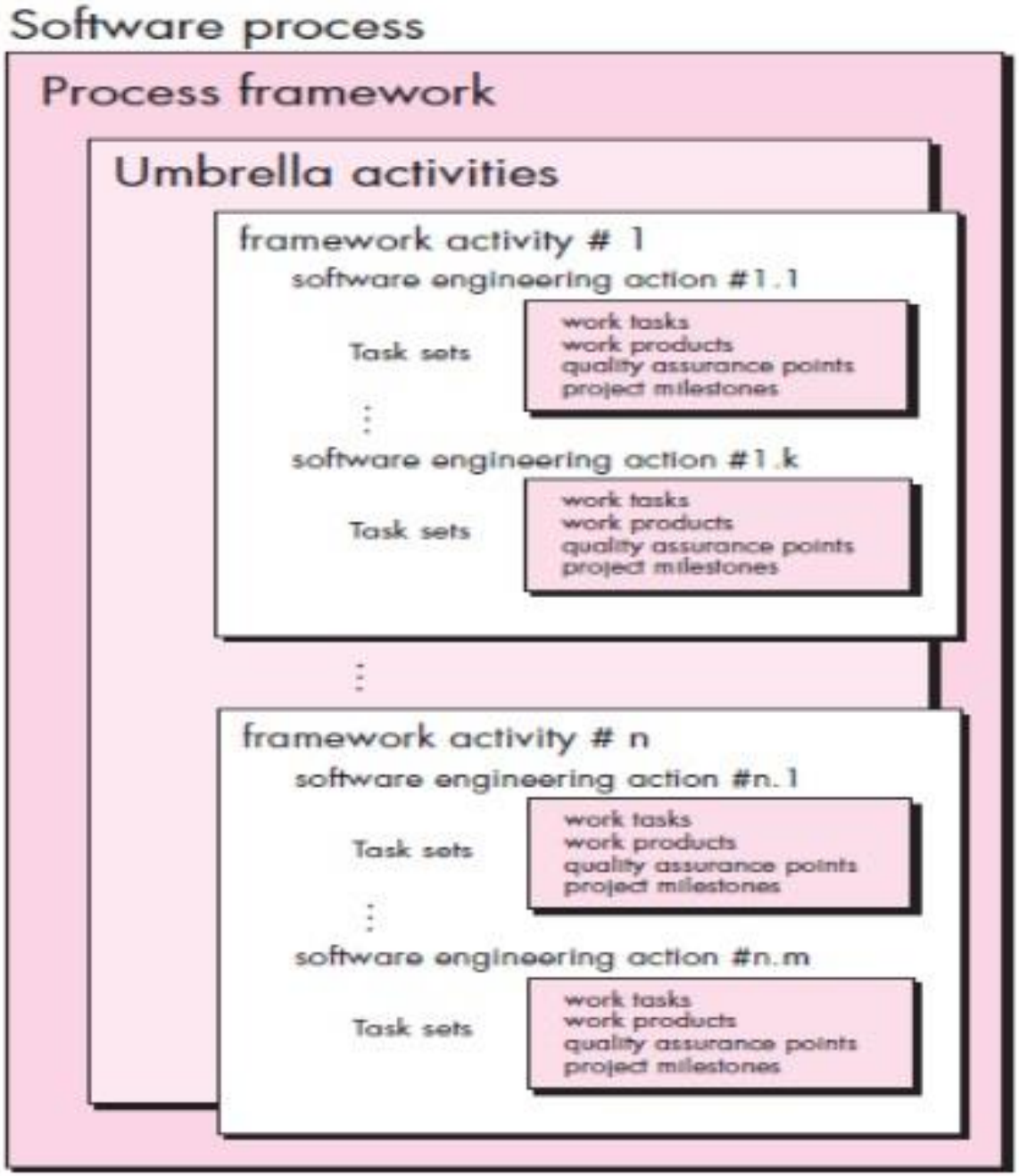
Evolutionary process flow

- **Parallel Process Flow:**A **parallel process flow** executes one or more activities in parallel with other activities



Parallel process flow

Software Process Framework



Generic Process Model

- For a small software project requested by one person (at a remote location) with simple, straightforward requirements, the communication activity might encompass little more than a phone call with the appropriate stakeholder. Therefore, the only necessary action is *phone conversation,* and the work tasks (the *task set*) that this action encompasses are:

1. Make contact with stakeholder via telephone.
2. Discuss requirements and take notes.
3. Organize notes into a brief written statement of requirements.
4. E-mail to stakeholder for review and approval.

Generic Process Model

Task Set: software engineering action can be represented by several different *task sets*.

For a small, relatively simple project, the task set for requirements gathering might look like this:

1. Make a list of stakeholders for the project.
2. Invite all stakeholders to an informal meeting.
3. Ask each stakeholder to make a list of features and functions required.
4. Discuss requirements and build a final list.
5. Prioritize requirements.
6. Note areas of uncertainty.

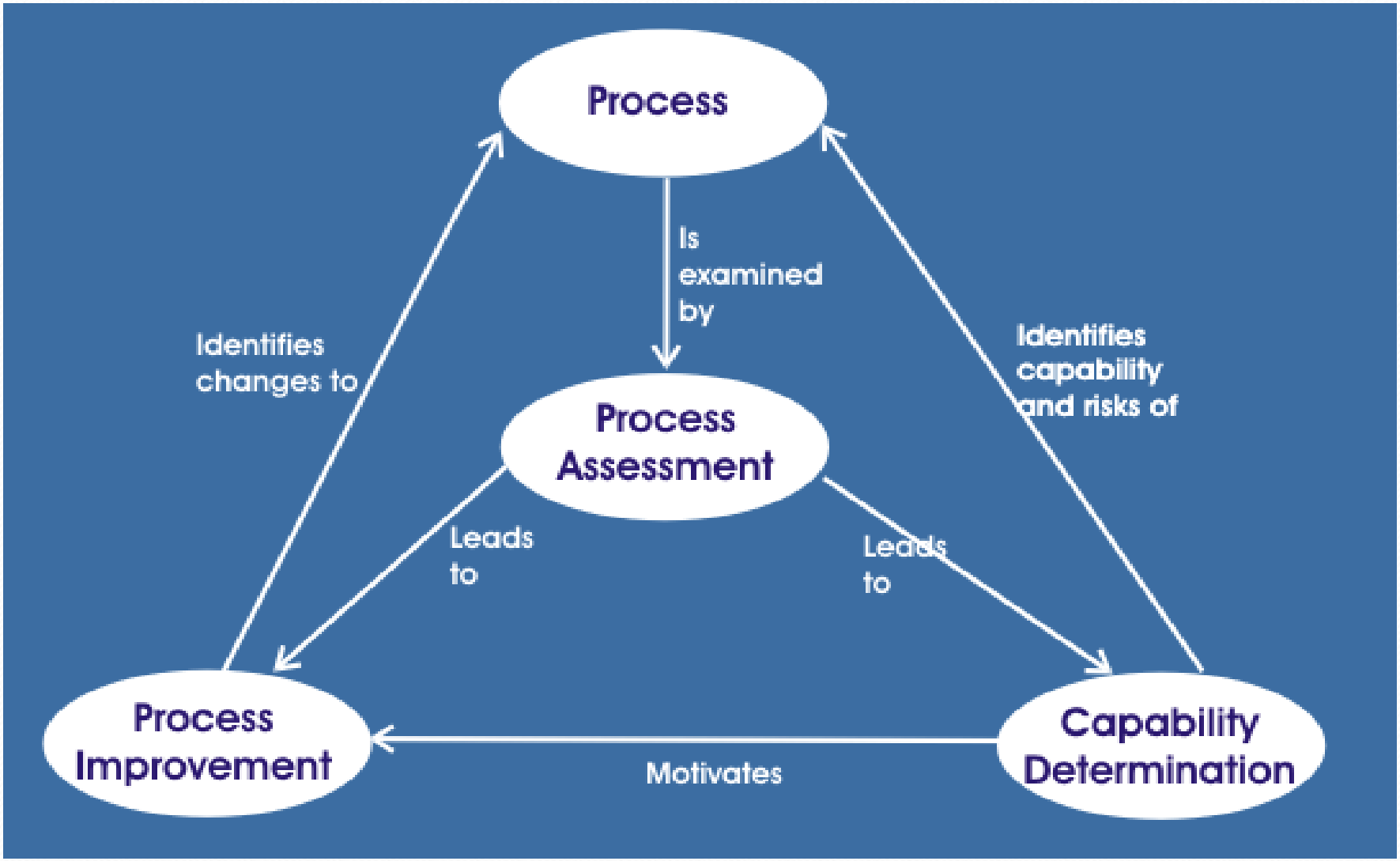
Process Assessment and Improvement

A software process assessment is a disciplined examination of the software processes used by an organization, based on a process model. The assessment includes the identification and characterization of current practices, identifying areas of strengths and weaknesses, and the ability of current practices to control or avoid significant causes of poor (software) quality, cost, and schedule.

A software assessment (or audit) can be of three types.

- A **self-assessment (first-party assessment)** is performed internally by an organization's own personnel.
- A **second-party assessment** is performed by an external assessment team or the organization is assessed by a customer.
- A **third-party assessment** is performed by an external party or (e.g., a supplier being assessed by a third party to verify its ability to enter contracts with a customer).

Software Assessment and Improvement



Prescriptive Process Model

- Definition of process Model.

  The process model can be defined as the abstract representation of process.

- The Software process model is also known as Software Development Life Cycle(SDLC)Model.

- These models are called prescriptive process model because they are following some rules for correct use.

- In this model various activities are carried out in some specific sequence to make the desired software product.

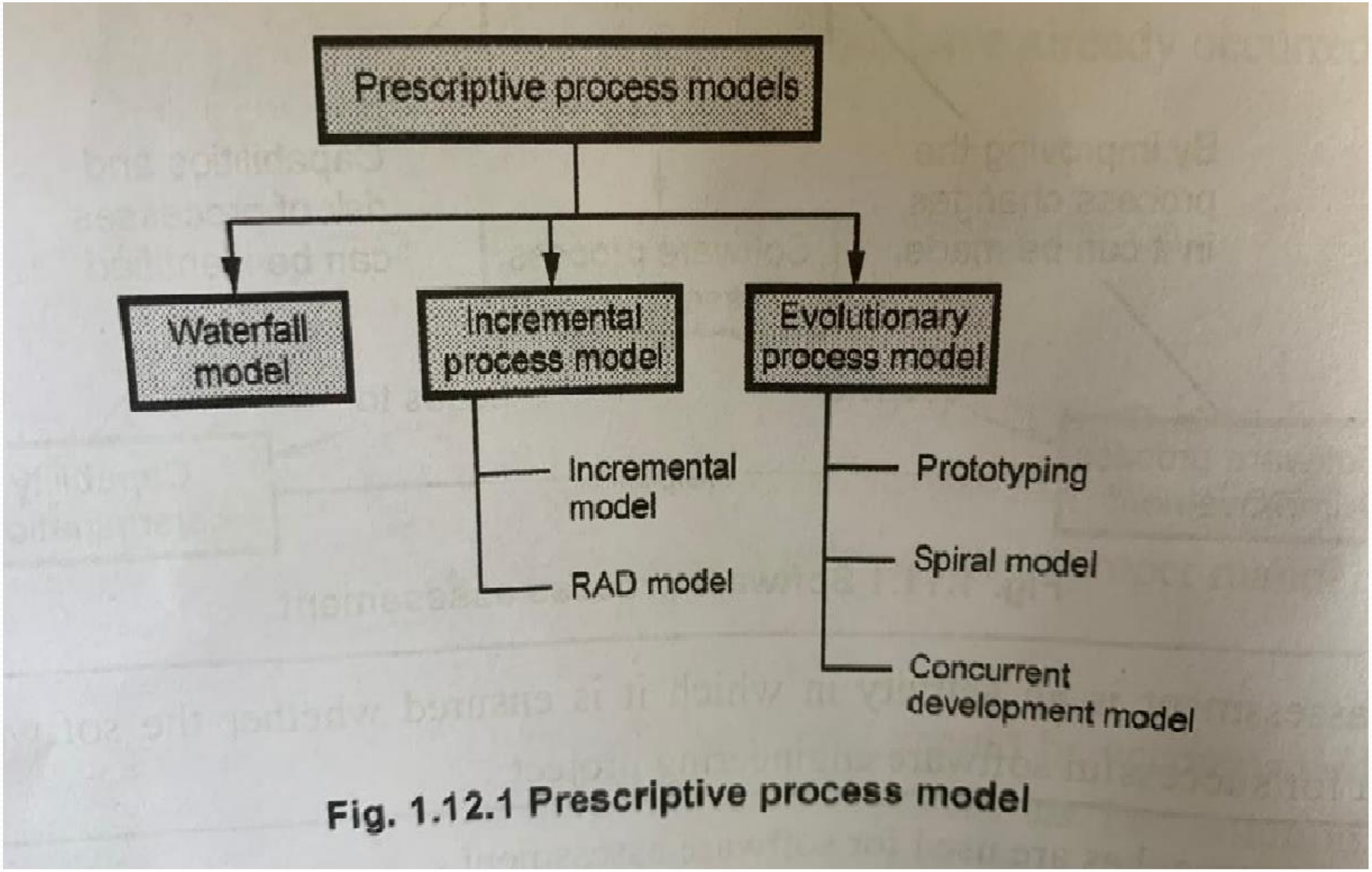


Fig. 1.12.1 Prescriptive process model

Need for Process Model

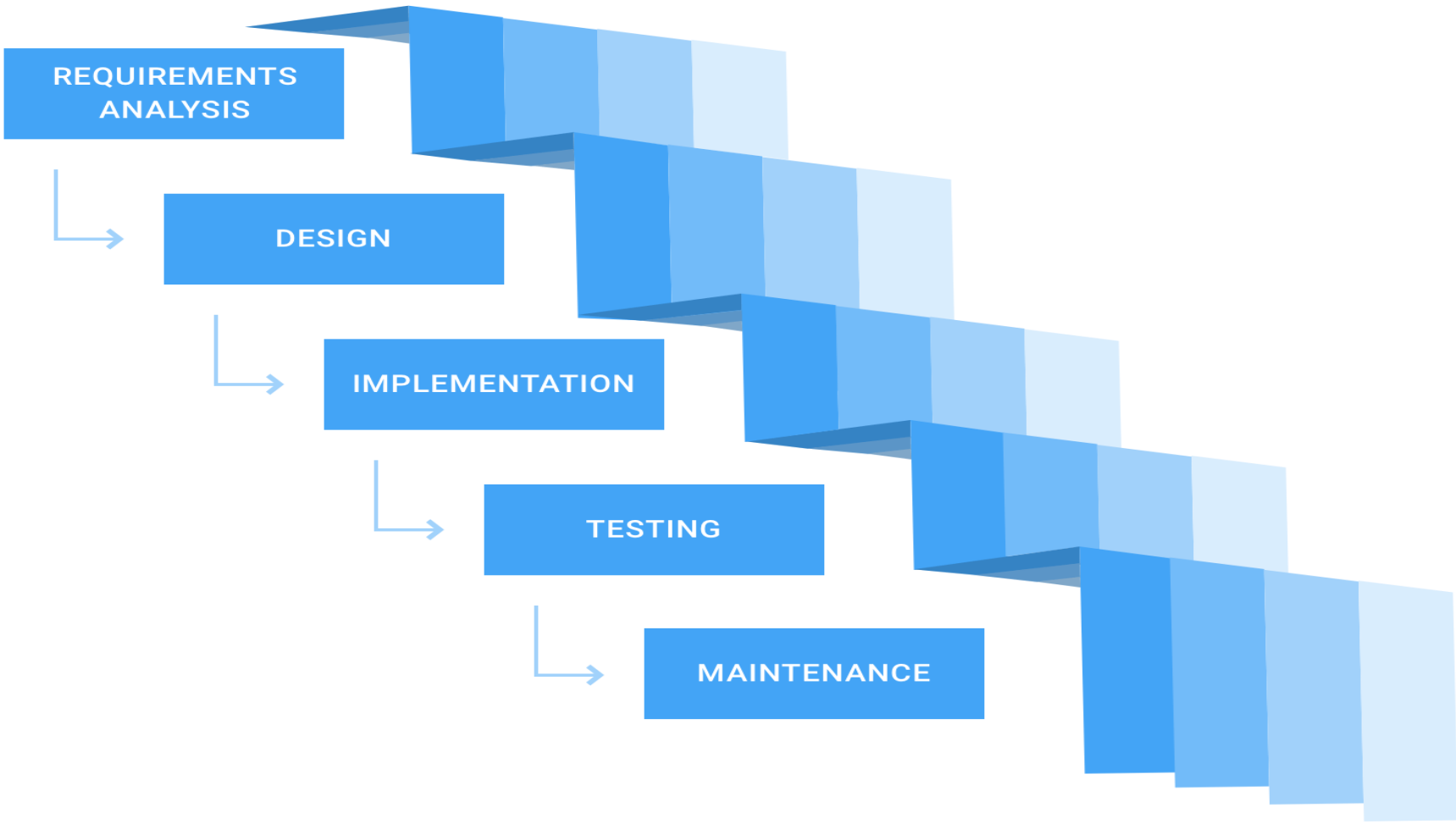
- Each team member will understand

  what is the next activity and how to do.
- The process model will bring the **definiteness and discipline** in overall development process.
- Every process model consists of **definite entry and exit criteria** for **each phase.**

Waterfall Model

- liner-sequential life cycle model or Classic life cycle model.
- No overlapping in phase.
- This model suggests a systematic, **sequential approach** to software development.
- The software development starts with requirements gathering phase.
- then progresses through **analysis , design, coding, testing and maintenance.**

Waterfall Model



Benefits of waterfall model

- Simple and easy.
- Easy to manage
- Works well for smaller projects.
- Results are well document.

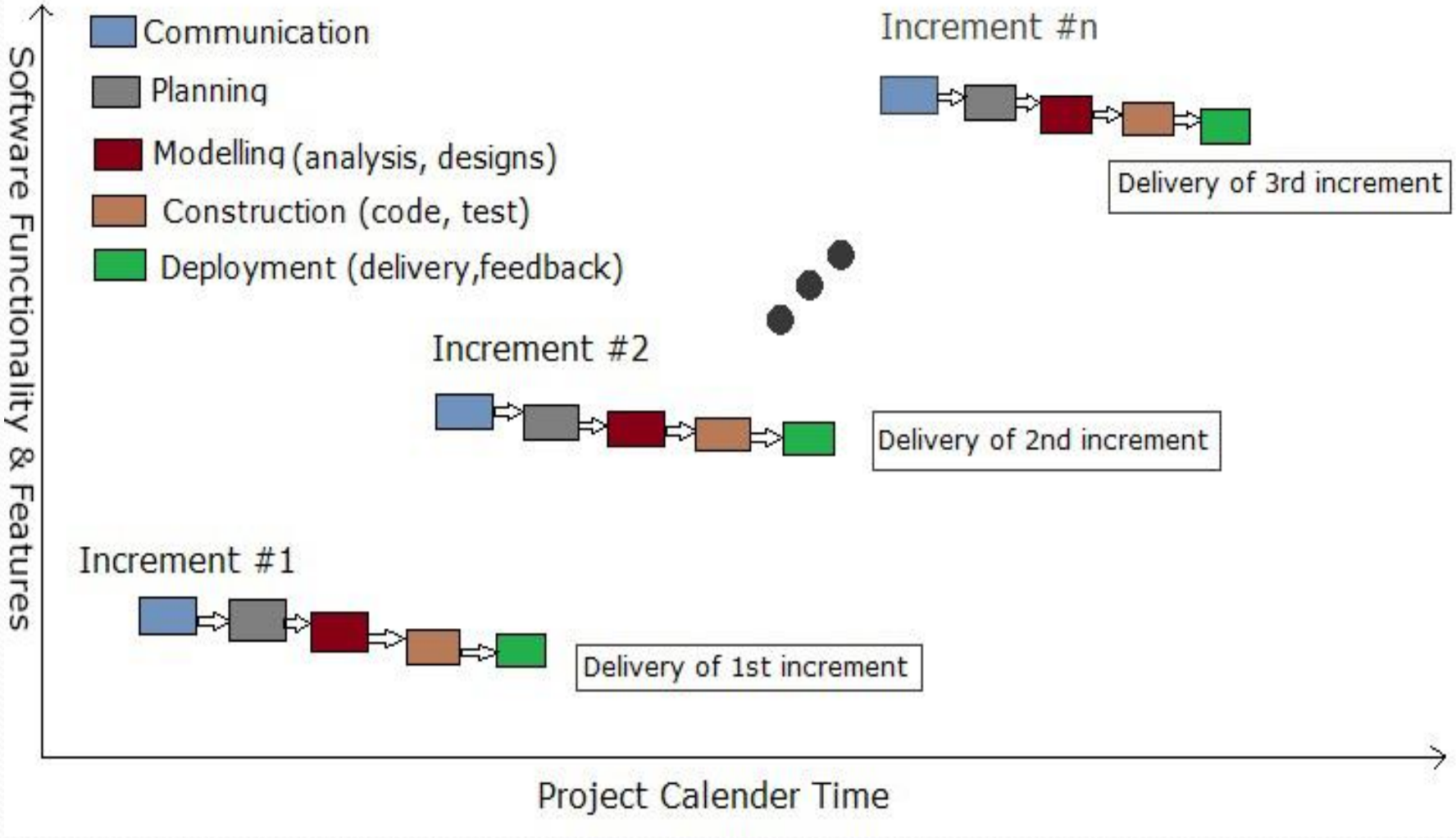
Drawbacks of waterfall model

- More risk.
- Not for big & complex projects.
- Not for projects where requirements are changing.
- Blocking states

Increment process model

- The incremental model delivers series of releases to the customer.
- This releases are called increments.
- More and more functionality is associated with each increment.
- The first increment is called core product.in this release the basic requirements are implemented and then in subsequent increments new requirements are added.
- The word processing software package can be considered as an example of increment model.

Fig. Increment model



When to Choose It?

- When requirements are reasonably well defined.
- When overall scope of development are limited.
- Project with new technology is required.

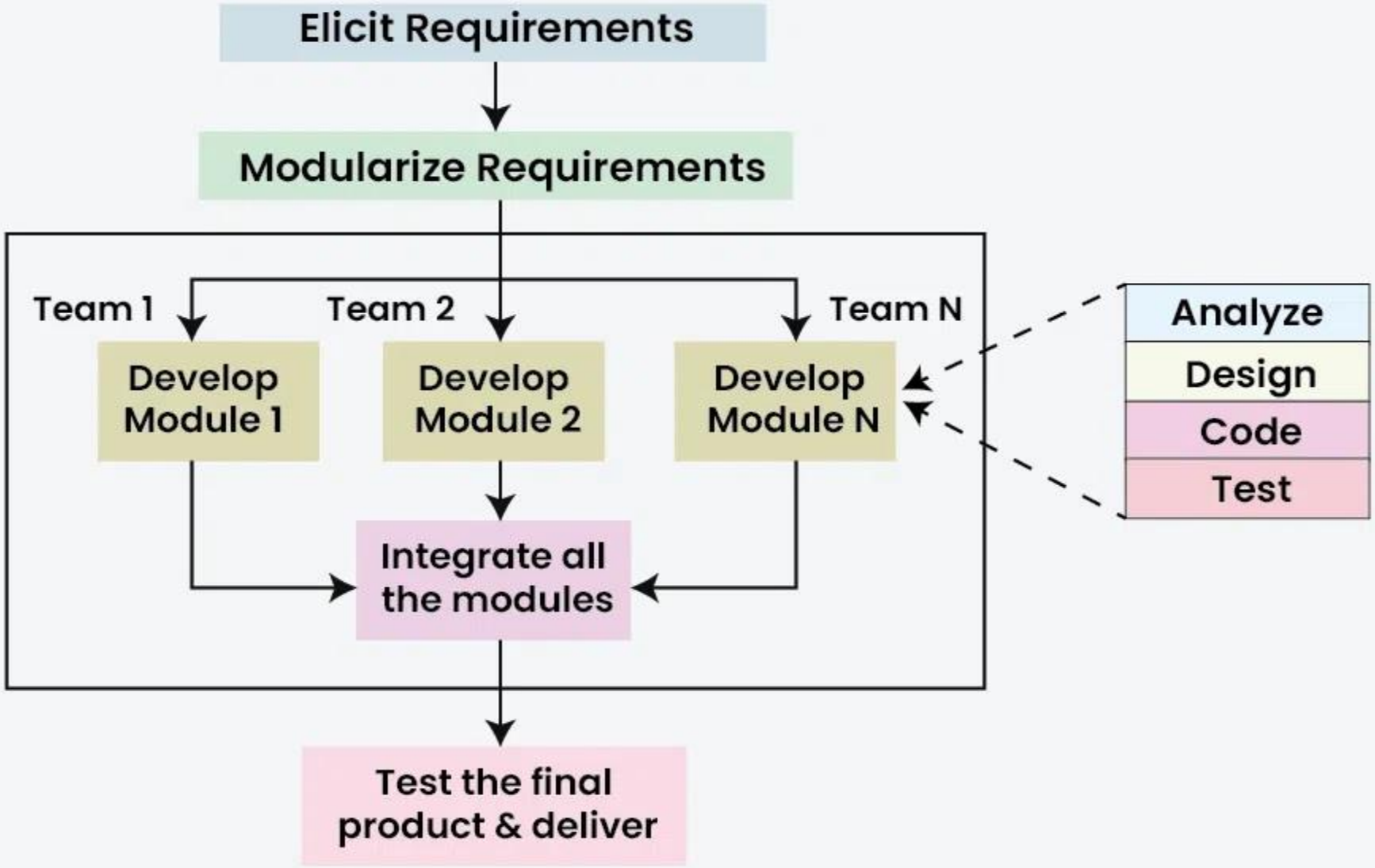
Advantages of Increment model

- Work with small size team.
- Initial product delivery is faster.
- Customer response/feedback is considered.

Disadvantages of Increment model

- Actual cost may exceed the estimated cost.
- System broken into small increments.

RAD Model


Elicit Requirements
Modularize Requirements
Team 1
Team 2
Team N
Develop Module 1
Develop Module 2
Develop Module N
Integrate all the modules
Test the final product & deliver
Analyze
Design
Code
Test

RAD Model:

- RAD (Rapid Application Development) is a concept that products can be developed faster and of higher quality through:
- Gathering requirements using workshops or focus groups
- Prototyping and early, reiterative user testing of designs
- The re-use of software components
- A rigidly paced schedule that refers design improvements to the next product version
- Less formality in reviews and other team communication

When to Use RAD Model:

- When the system should need to create the project that modularizes in a short span time (2-3 months).
- When the requirements are well-known.
- When the technical risk is limited.
- When there's a necessity to make a system, which modularized in 2-3 months of period.
- It should be used only if the budget allows the use of automatic code generating tools.

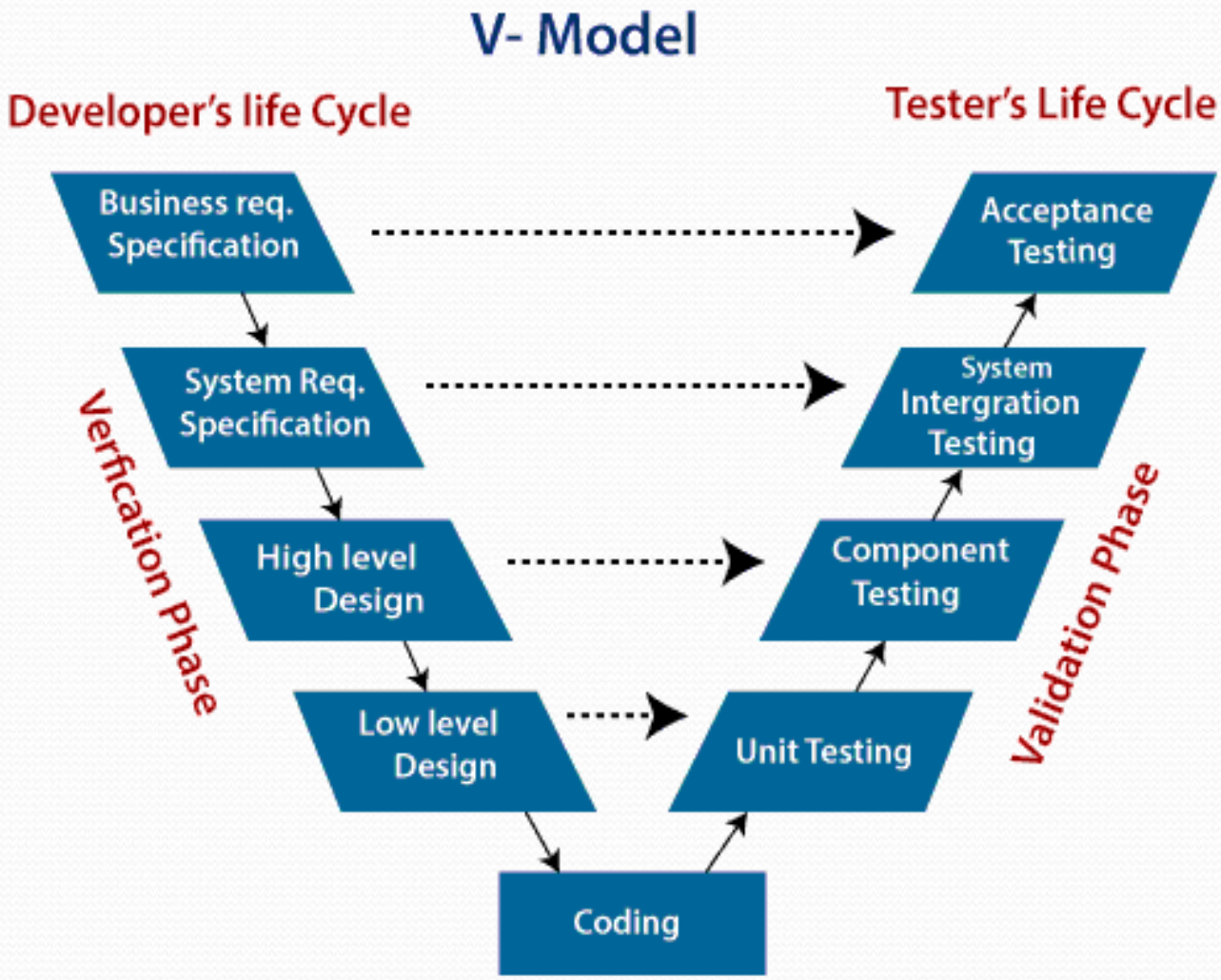
Advantages of RAD model

- This model is flexible for change.
- In this model, changes are adoptable.
- Each phase in RAD brings highest priority functionality to the customer.
- It reduced development time.
- It increases the reusability of features.

Disadvantages:

- It can't be used for smaller projects.
- When technical risk is high, it is not suitable.
- When developers are not committed to delivering software on time, RAD projects can fail.

V Model:


V- Model
Developer's life Cycle
Tester's Life Cycle
Business req. Specification
System Req. Specification
High level Design
Low level Design
Coding
Acceptance Testing
System Intergration Testing
Component Testing
Unit Testing
Verfication Phase
Validation Phase

Evolutionary Process Models:

- Prototyping Model
- Spiral Model
- Concurrent Model

Prototyping model

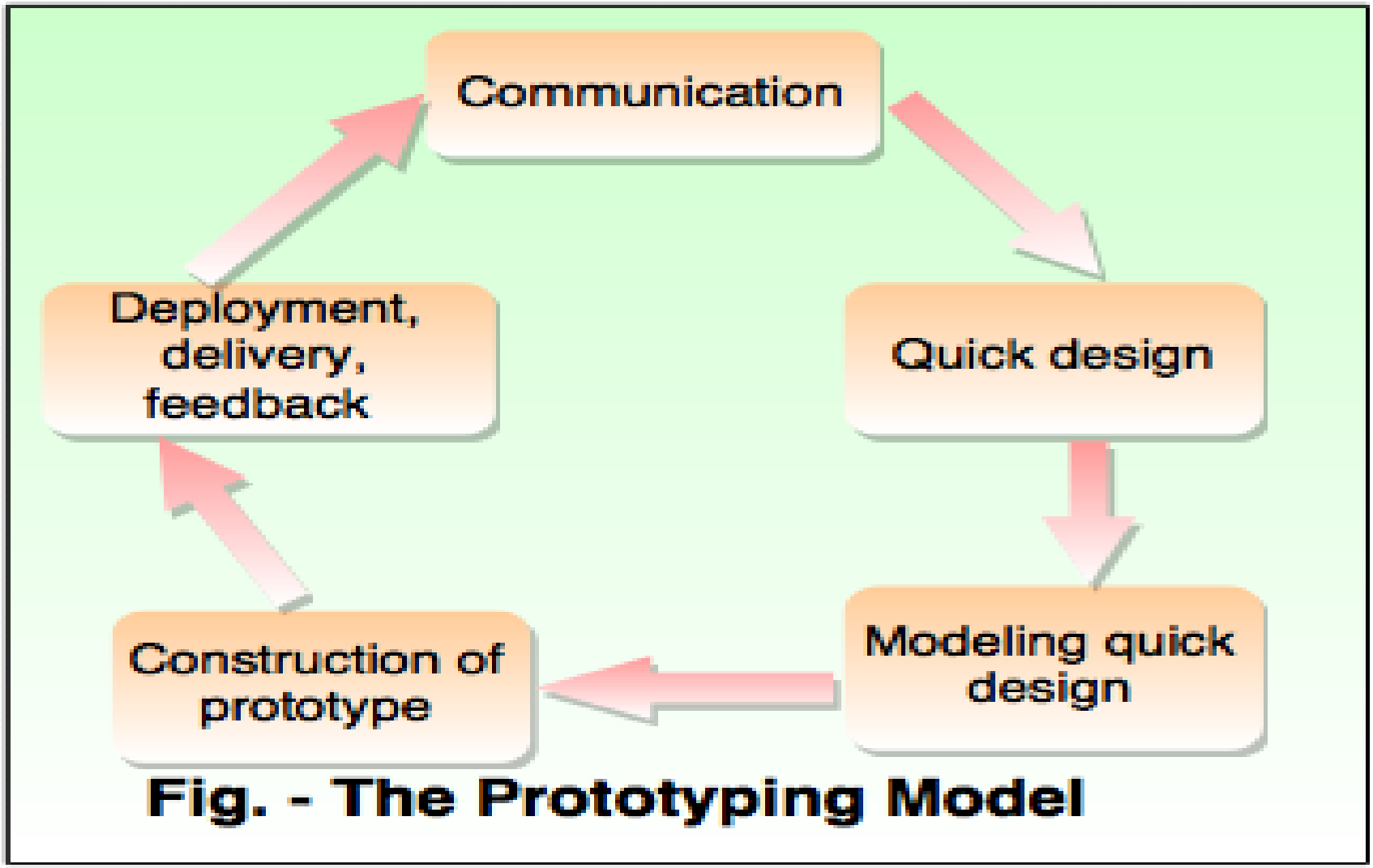


Fig. - The Prototyping Model

What is Software prototyping

Prototype is a working model of software with some limited functionality. The prototype does not always hold the exact logic used in the actual software application and is an extra effort to be considered under effort estimation.

When to use Prototype model:

- Prototype model should be used when the desired system needs to have a lot of interaction with the end users.
- Typically, online systems, web interfaces have a very high amount of interaction with end users, are best suited for Prototype model.

Advantages and Disadvantages of Prototype Model

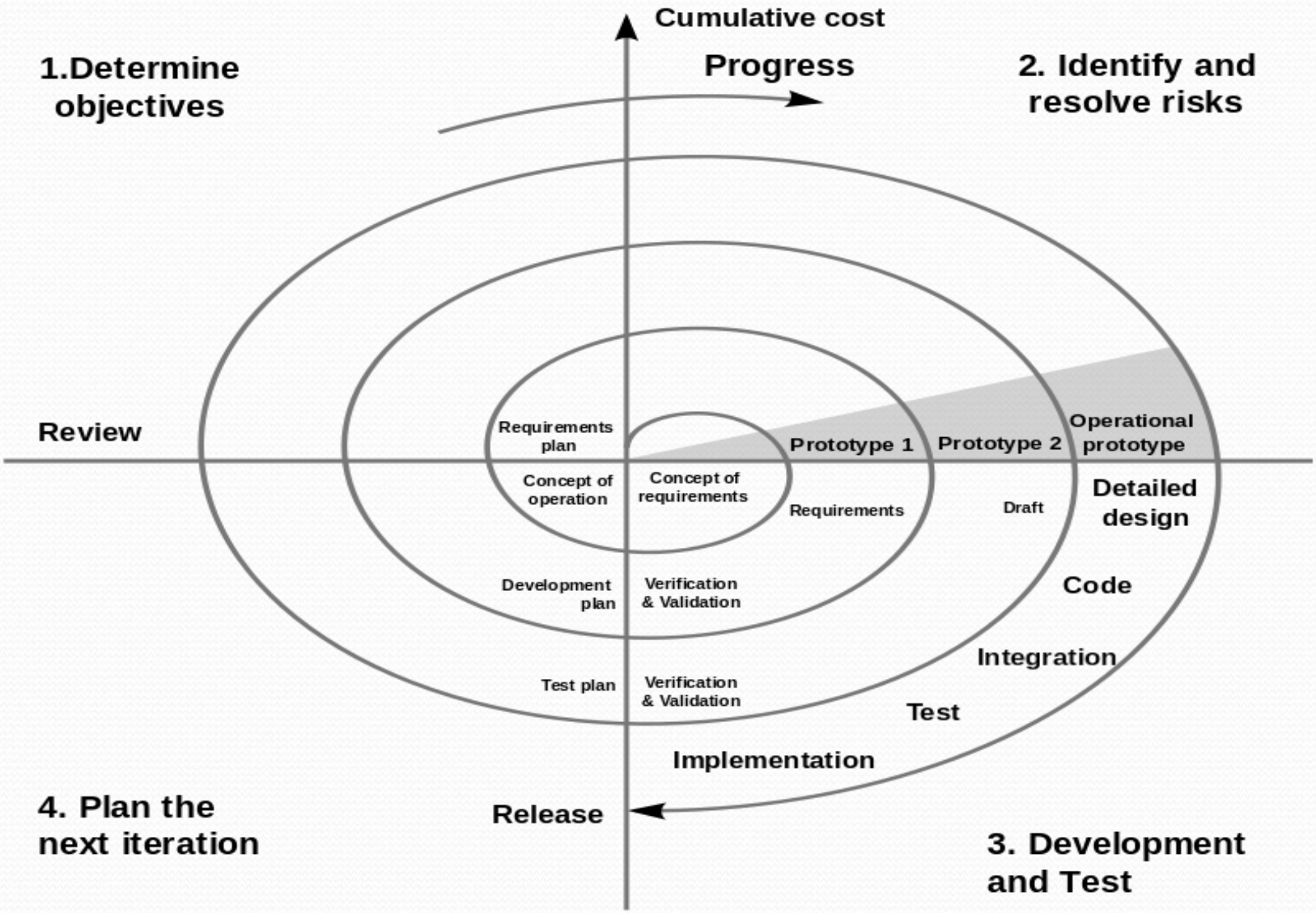
Advantages of Prototype model:

- Users are actively involved in the development
- Since in this methodology a working model of the system is provided, the users get a better understanding of the system being developed.
- Errors can be detected much earlier.

Disadvantages of Prototype model:

- Leads to implementing and then repairing way of building systems.
- Practically, this methodology may increase the complexity of the system as scope of the system may expand beyond original plans.

Spiral Model



Spiral Model

- **Spiral model** is one of the most important Software Development Life Cycle models, which provides support for **Risk Handling**.
- Each loop of the spiral is called a **Phase of the software development process.**

The functions of these four quadrants are discussed below-

1)Objectives determination and identify alternative solutions.

2)Identify and resolve Risks

3) Develop next version of the Product

4) Review and plan for the next Phase

When to choose Spiral Model?

- When the prototype for the software functionality are needed.
- When the **large high budget** projects are developed.
- When the risk assessment is very critical and essential.

Advantages and Disadvantaes of Spiral Model

Advantages of Spiral Model:

i) Below are some advantages of the Spiral Model.

ii) Risk Handling

iii) Good for large projects

iv) Flexibility in Requirements

v) Customer Satisfaction

Disadvantages of Spiral Model:

i) Complex

ii) Expensive

iii)Too much dependability on Risk Analysis

iv) Difficulty in time management

Concurrent Model

- The Concurrent development model is also called **Concurrent Engineering.**

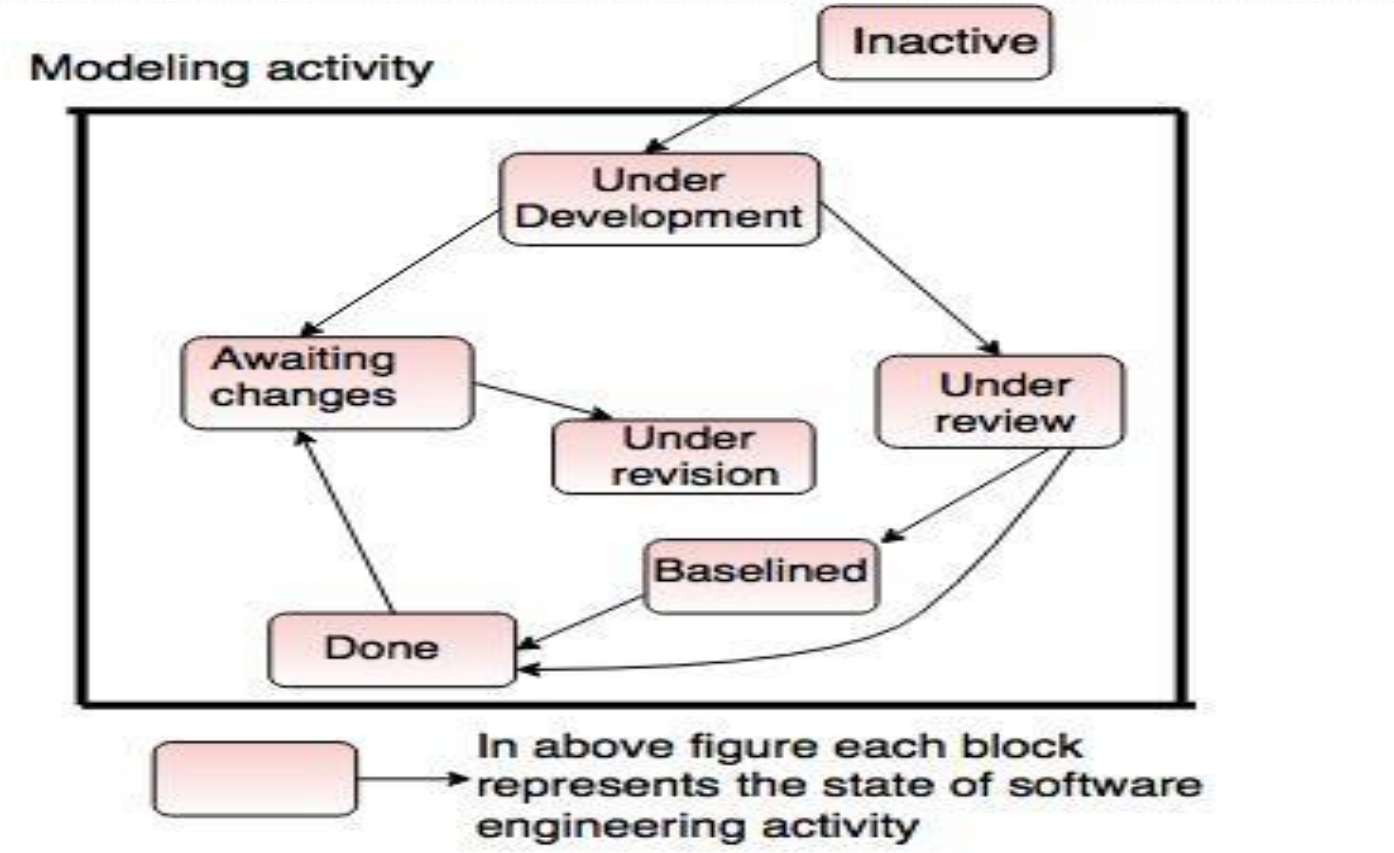


Fig. - One element of the concurrent process model

Advantages

- All type of software development can be done using concurrent models.
- This model provides accurate picture of current state of project.
- Each activity or task can be carried out concurrently

Hence, this model is an **efficient process model.**

Agile Software Development

Agile Process: The aim of agile process is to deliver working model quickly to the customer.

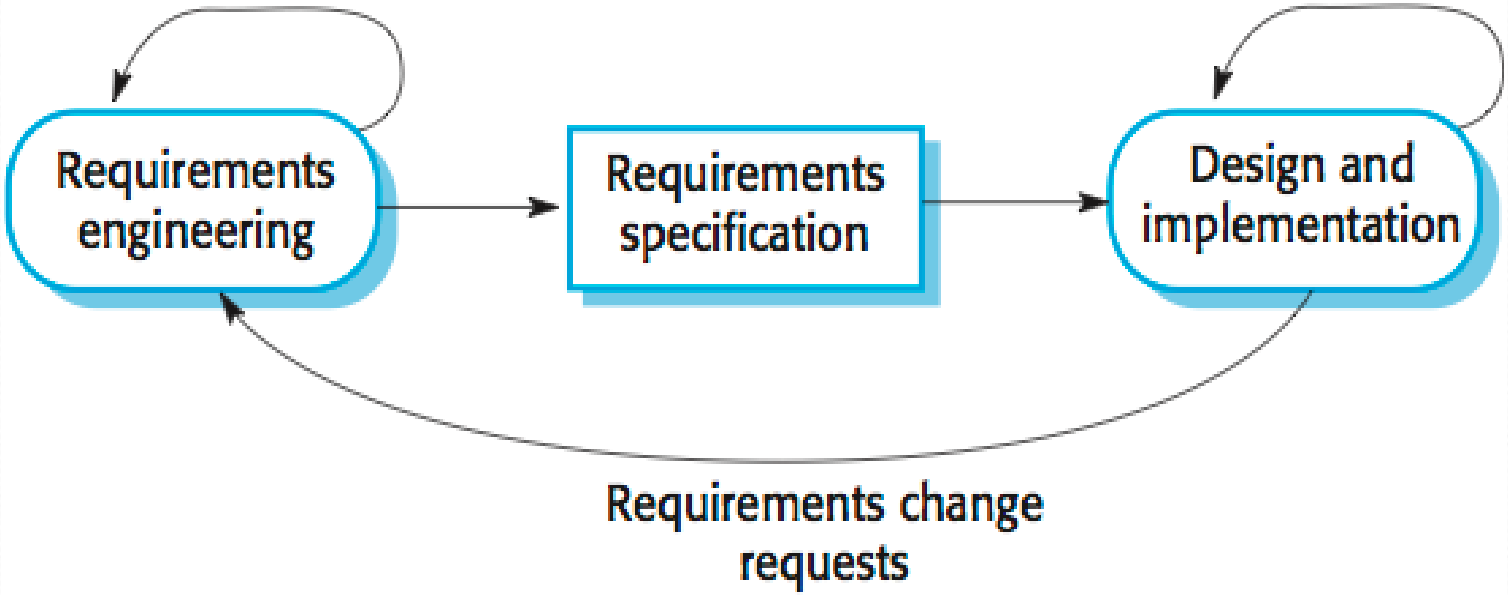
i)It must be adaptable to technical and environmental changes.

ii)The development of agile process must be incremental.

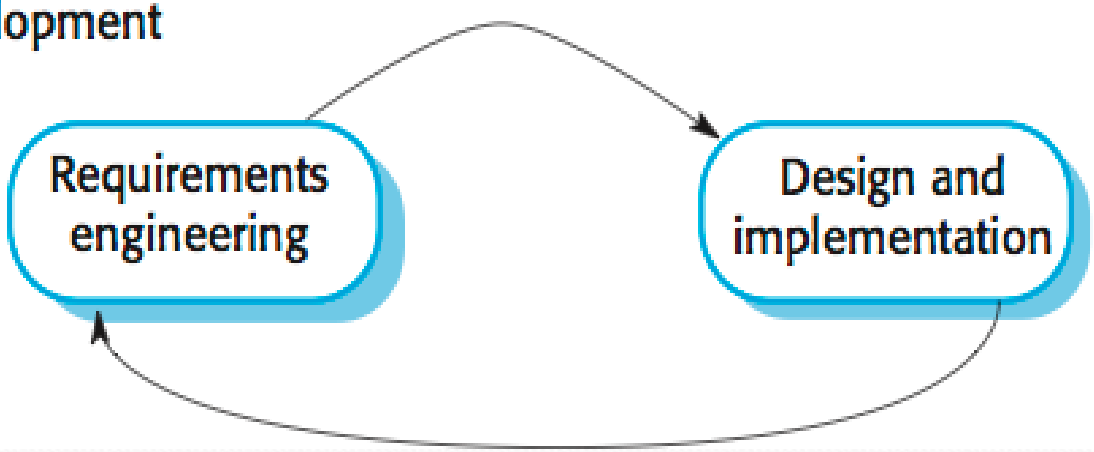
iii)The Customer feedback must be used to create the next increment of the process.

iv)The software must be deliver in a short **span of time.**

Plan-based development



Agile development



Agile Manifesto

The 4 Core Values of the Agile Manifesto

The manifesto states that while there is value in the items on the right, the items on the left are valued more:

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

The 12 Guiding Principles of Agile Manifesto

These principles provide practical guidance for implementing the values:

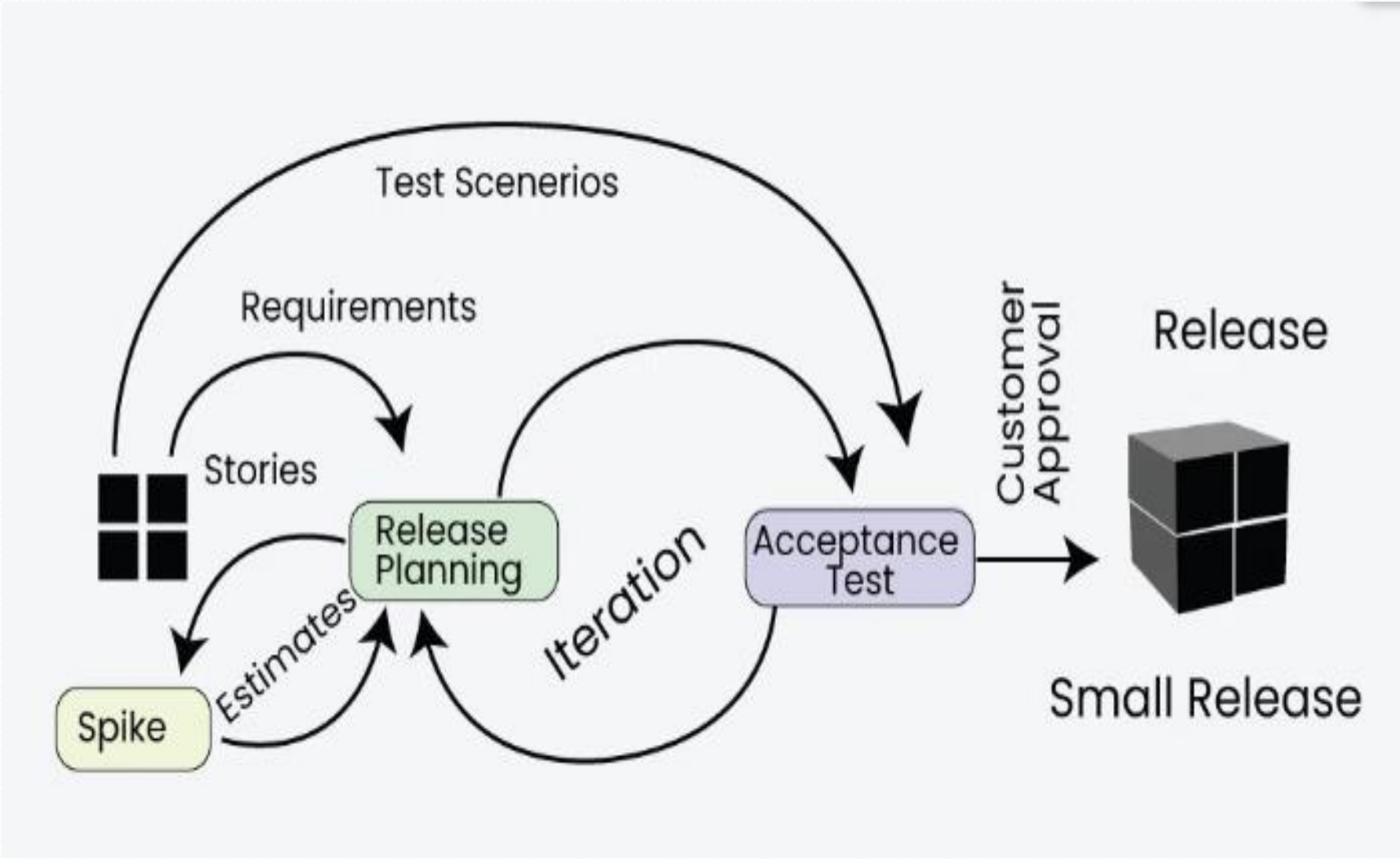
- **Customer Satisfaction**: Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
- **Welcome Change**: Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
- **Frequent Delivery**: Deliver working software frequently, from a couple of weeks to a couple of months, with a preference for shorter timescales.
- **Daily Collaboration**: Business people and developers must work together daily throughout the project.
- **Motivated Individuals**: Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.

- **Face-to-Face Conversation**: The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
- **Working Software as Progress**: Working software is the primary measure of progress.
- **Sustainable Development**: Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- **Technical Excellence**: Continuous attention to technical excellence and good design enhances agility.
- **Simplicity**: Simplicity—the art of maximizing the amount of work not done—is essential.
- **Self-Organizing Teams**: The best architectures, requirements, and designs emerge from self-organizing teams.
- **Reflect and Adjust**: At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly

EXTREME PROGRAMMING (XP)

- Extreme Programming (XP) is a key Agile framework focused on building high-quality software while quickly adapting to changing requirements. It strengthens proven development practices by applying them in a more disciplined and consistent way.
- XP promotes continuous feedback, close collaboration with customers, and short development cycles. It follows an iterative approach where software is developed, tested, and improved regularly, ensuring better quality and faster delivery.

EXTREME PROGRAMMING (XP)



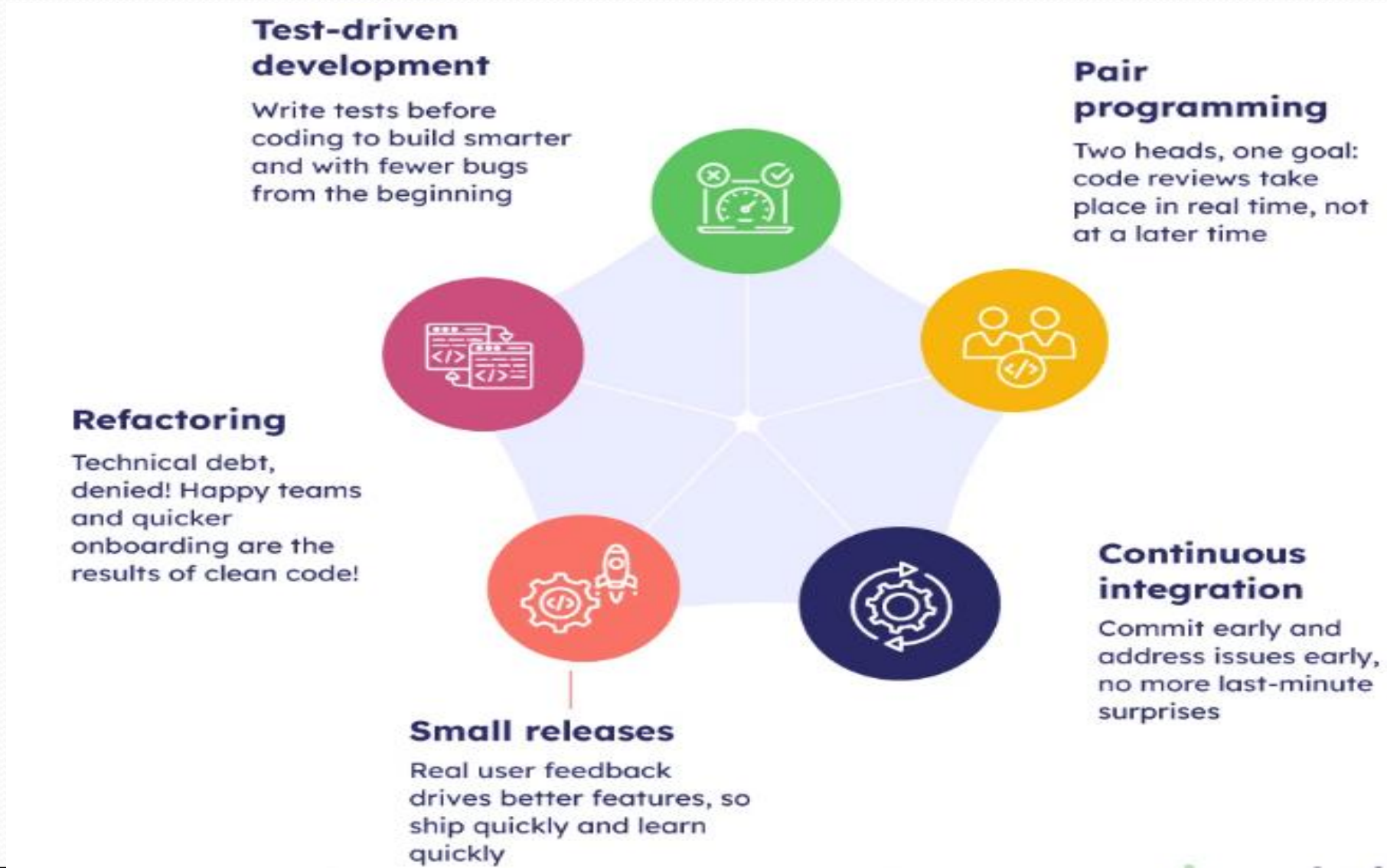
EXTREME PROGRAMMING (XP)

The 5 Core Values of XP

- These are the cultural foundation of XP, going beyond the Agile Manifesto:
- **Communication** – Face-to-face, daily, and honest dialogue.
- **Simplicity** – Build only what is needed *right now* ("Do the simplest thing that could possibly work").
- **Feedback** – Get constant feedback from tests, customers, and the code itself.
- **Courage** – Have the guts to refactor code, throw away bad work, and push back on unrealistic deadlines.
- **Respect** – Respect your teammates, the customer, and the codebase.

EXTREME PROGRAMMING (XP)

The 5 Key Practices of XP



EXTREME PROGRAMMING (XP)

The 5 Key Practices of XP

1. **TDD:** Write tests before code → ensures high coverage.
2. **Pair Programming:** Two devs at one station → continuous review.
3. **Continuous Integration:** Integrate code multiple times daily → catches conflicts early.
4. **Refactoring:** Continuously improve internal structure → keeps code maintainable.
5. **Collective Ownership:** Anyone can change any code → eliminates bottlenecks.